

UNIVERSITÉ GASTON BERGER DE SAINT-LOUIS
UFR DE SCIENCES APPLIQUÉES ET DE TECHNOLOGIE
CENTRE DE FORMATION PROFESSIONNELLE PLURIDISCIPLINAIRE (CFPP)
LICENCE PROFESSIONNELLE GÉNIE INFORMATIQUE (LPGI2)

Rapport de Projet Professionnel

Sujet :

Optimisation des performances d'une base de données Oracle
Étude de cas appliquée au système d'information de DistriPlus Group

Présenté et soutenu par :
Mamadou TOURE et Mamadou SECK

Sous la direction de : Mr Cyril ANGELIER LEPLAN

Année académique 2025/2026

Sommaire

Ce rapport présente le travail réalisé dans le cadre d'un projet professionnel de fin de deuxième année de formation de Licence Professionnelle en Génie Informatique (LPGI 2), portant sur l'optimisation des performances d'une base de données Oracle. L'étude s'appuie sur un cas concret librement inspiré de l'activité d'une entreprise de distribution, DistriPlus Group, pour laquelle nous avons conçu, peuplé puis optimisé une base de données représentative de son activité commerciale (gestion des ventes, des stocks et des approvisionnements).

L'objectif du projet est double. D'une part, comprendre et appliquer les outils de diagnostic propres à Oracle — EXPLAIN PLAN et AUTOTRACE — afin d'identifier précisément l'origine d'une lenteur d'exécution. D'autre part, mettre en œuvre puis mesurer des techniques d'optimisation relevant de deux axes distincts : l'axe applicatif, qui agit sur les requêtes et le code PL/SQL sans modifier la structure des tables, et l'axe modélisation, qui agit au contraire sur cette structure elle-même, par le biais de choix de dénormalisation contrôlée.

Quatre cas d'optimisation ont été traités, deux par axe, chacun mesuré avant et après intervention à l'aide d'indicateurs chiffrés (coût d'exécution, nombre de lectures logiques, temps réel d'exécution). Les résultats obtenus, présentés en détail dans les chapitres 3 et 4, montrent des gains allant de 31 % à 95 % selon le cas considéré, avec pour chaque optimisation une explication du mécanisme sous-jacent propre au fonctionnement de l'optimiseur Oracle.

Ce travail nous a permis de développer une compréhension concrète du fonctionnement interne d'un système de gestion de base de données relationnelle, au-delà de la simple écriture de requêtes fonctionnelles, et de nous familiariser avec une démarche d'ingénierie fondée sur la mesure plutôt que sur l'intuition.

Remerciements

Nous tenons à remercier en premier lieu Mr ANGELIER, notre enseignant encadrant, pour la disponibilité et la rigueur dont il a fait preuve tout au long de ce projet. Ses remarques, notamment sur la distinction entre optimisation applicative et optimisation de modélisation, nous ont permis de structurer notre démarche de façon beaucoup plus rigoureuse.

Nous remercions également l'ensemble de l'équipe pédagogique de la Licence Professionnelle Génie Informatique de l'Université Gaston Berger, pour la qualité des enseignements en bases de données et en systèmes qui ont rendu ce projet possible.

Enfin, nous remercions nos camarades de promotion pour les échanges techniques qui ont accompagné ce travail.

Table des matières

Sommaire	2
Remerciements	3
Table des matières	4
Liste des tableaux	6
Liste des figures	6
Introduction	7
Contexte	7
Objectifs	7
Problématique.....	7
Méthodologie et outils	8
Structure du rapport.....	8
Chapitre 1 — Présentation du projet et de son cadre technique.....	9
1.1 Contexte et choix technologiques	9
1.2 Modélisation de la base de données	9
1.3 Génération du jeu de données de test.....	10
1.4 Vérification préalable de la base.....	12
Chapitre 2 — Méthodologie de diagnostic des performances.....	13
2.1 La collecte des statistiques.....	13
2.2 Le plan d'exécution avec EXPLAIN PLAN.....	13
2.3 La mesure réelle avec AUTOTRACE	14
2.4 Une méthode complémentaire : le chronométrage direct.....	14
Chapitre 3 — Optimisation applicative	16
3.1 Cas 1 — La requête du tableau de bord des ventes.....	16
3.1.1 État initial et diagnostic	16
3.1.2 Reformulation de la requête.....	16
3.1.3 Création d'un index composite	17
3.1.4 Résultats et discussion	17
3.2 Cas 2 — La procédure de valorisation du stock.....	18
3.2.1 Version initiale.....	18
3.2.2 Version optimisée par traitement ensembliste	19
3.2.3 Résultats et discussion	19
Chapitre 4 — Optimisation de la modélisation	21
4.1 Cas 1 — Dénormalisation contrôlée du prix produit	21
4.1.1 Comparaison avec et sans dénormalisation	21
4.1.2 Discussion	22

4.2 Cas 2 — Redondance contrôlée d'une clé étrangère	22
4.2.1 Requête avant modification	23
4.2.2 Ajout de la clé étrangère redondante.....	23
4.2.3 Résultats et discussion	23
Chapitre 5 — Synthèse comparative des optimisations	25
5.1 Tableau de synthèse.....	25
5.2 Enseignements transversaux.....	25
Conclusion et perspectives.....	26
Bilan de la problématique.....	26
Ce qu'il reste à approfondir.....	26
Compétences acquises	27
Perspectives	27
Annexe A — Script complet de création du schéma	27
Annexe B — Script de génération du jeu de données.....	28
Lexique.....	29
Bibliographie — Webographie	30

Liste des tableaux

- Tableau 1 — Volumes de données générés pour chaque table du schéma
- Tableau 2 — Le facteur de regroupement (clustering factor)
- Tableau 3 — Évolution du coût et des lectures logiques pour le Cas 1
- Tableau 4 — Comparaison des temps d'exécution pour le Cas 2
- Tableau 5 — Comparaison des temps d'exécution pour le Cas 1 de modélisation
- Tableau 6 — Comparaison avant/après pour le Cas 2 de modélisation
- Tableau 7 — Synthèse des quatre cas d'optimisation traités

Liste des figures

- Figure 1 — Schéma relationnel simplifié des sept tables du projet
- Figure 2 — Extrait du script de génération de la table COMMANDE (500 000 lignes)
- Figure 3 — Collecte des statistiques du schéma DISTRIPUS
- Figure 4 — Plan d'exécution de la requête initiale (extrait)
- Figure 5 — Requête reformulée avec filtre sur le magasin et la période
- Figure 6 — Création de l'index composite
- Figure 7 — Procédure de valorisation du stock — version initiale
- Figure 8 — Procédure de valorisation du stock — version optimisée
- Figure 9 — Requête avec dénormalisation (prix déjà présent dans LIGNE_COMMANDE)
- Figure 10 — Requête équivalente sans dénormalisation (jointure avec PRODUIT)
- Figure 11 — Requête nécessitant une jointure transitive via COMMANDE
- Figure 12 — Ajout de la colonne redondante magasin_id
- Figure 13 — Requête après ajout de la clé étrangère redondante
- Figure 14 — Création des sept tables du schéma
- Figure 15 — Génération de la table STOCK (600 000 lignes)

Introduction

Contexte

Une base de données relationnelle bien conçue ne garantit pas, à elle seule, de bonnes performances à l'usage. Passé un certain volume de données, une requête syntaxiquement correcte peut devenir lente au point de rendre une application difficilement utilisable, sans que la logique métier qu'elle exprime soit en cause : c'est précisément ce constat qui justifie la pertinence de ce projet car une erreur de diagnostic peut engendrer des coûts importants et inutiles pour l'ensemble des utilisateurs et décideurs de l'entreprise qui utilisent cette application.

Dans le cadre de l'exploration d'une telle situation pour en comprendre les raisons afin de décider des mesures correctives les plus appropriées à mettre en place, nous ne nous sommes pas limités à la conception d'un schéma de base données. Nous sommes passés à l'étape suivante d'interrogation de la base afin d'identifier l'origine des opérations coûteuses.

Nous avons construit pour cela un cas d'usage librement inspiré d'une entreprise de distribution, que nous avons nommée DistriPlus Group. Cette entreprise fictive gère un réseau de magasins, un catalogue de produits fournis par des fournisseurs, ainsi que les ventes et les stocks associés. Ce contexte, bien que simplifié par rapport à un système d'information réel, présente l'avantage de rassembler les situations les plus courantes en matière d'optimisation de bases de données commerciales : requêtes de reporting, procédures de calcul en masse, choix de modélisation impliquant des compromis entre lecture et écriture.

Objectifs

Ce projet poursuit deux objectifs complémentaires. Le premier est méthodologique : apprendre à diagnostiquer une requête ou une procédure lente à l'aide des outils fournis par Oracle, en particulier EXPLAIN PLAN, qui permet d'observer le plan d'exécution prévue par l'optimiseur, et AUTOTRACE, qui permet de mesurer l'exécution réelle d'une instruction. Le second est pratique : mettre en œuvre des techniques d'optimisation concrètes, relevant de deux axes distincts que le sujet du projet impose de traiter séparément — l'axe applicatif, qui consiste à améliorer les performances par des moyens n'affectant pas la structure des tables (réécriture de requête, création d'index, réécriture de procédure PL/SQL), et l'axe modélisation, qui consiste au contraire à faire évoluer cette structure elle-même, par des choix de dénormalisation contrôlée.

Problématique

La question à laquelle ce rapport tente de répondre peut se formuler ainsi : dans quelle mesure une modification ciblée, soit du code applicatif, soit du modèle de données, permet-elle d'améliorer mesurablement les performances d'une base de données Oracle sur des volumes représentatifs d'une activité réelle, et à quel prix en termes de complexité ou de risque d'incohérence ?

Cette question suppose de traiter chaque optimisation non comme une recette isolée, mais comme un compromis dont il faut être capable d'expliquer le mécanisme : pourquoi une requête était lente, ce que l'on a changé, et pourquoi ce changement a (ou n'a pas) produit l'effet attendu.

Méthodologie et outils

L'ensemble du projet a été réalisé sur une instance Oracle Database 11g Express Edition installée localement, pilotée par l'interface graphique SQL Developer et, pour certaines vérifications, par SQL*Plus en ligne de commande. Aucun outil de diagnostic autre qu'EXPLAIN PLAN et AUTOTRACE n'a été utilisé, ce choix ayant été explicitement demandé par notre encadrant afin de nous concentrer sur la maîtrise de ces deux outils plutôt que de multiplier les approches.

Pour chaque cas d'optimisation, la même démarche a été suivie : mesure de l'état initial, identification de la cause de la lenteur observée, mise en œuvre d'une modification précise, puis nouvelle mesure dans les mêmes conditions. Lorsque le coût d'exécution ne permettait pas de distinguer clairement les deux situations, ce qui s'est produit lorsque la table jointe était de très petite taille, nous avons complété la mesure par un chronométrage réel de l'exécution, à l'aide de la commande SET TIMING.

Un jeu de données représentatif a été généré pour chaque table du schéma, avec des volumes volontairement importants (jusqu'à un million de lignes pour la table des lignes de commande) afin que les effets mesurés soient représentatifs d'un contexte de production et non de simples artefacts liés à un jeu de test trop restreint.

Structure du rapport

Le chapitre 1 présente le contexte métier retenu, le schéma relationnel conçu pour DistriPlus Group ainsi que la démarche de génération du jeu de données de test. Le chapitre 2 détaille la méthodologie de diagnostic utilisée tout au long du projet. Les chapitres 3 et 4 constituent le cœur du mémoire : ils présentent respectivement les deux cas de l'axe applicatif et les deux cas de l'axe modélisation, avec pour chacun l'état initial, la modification apportée et les résultats obtenus. Le chapitre 5 propose une synthèse comparative des quatre cas. Le mémoire se conclut par un bilan critique du travail réalisé et par un lexique regroupant les principales notions techniques employées.

Chapitre 1 — Présentation du projet et de son cadre technique

Ce premier chapitre présente le contexte métier retenu pour le projet, les choix technologiques effectués, le schéma relationnel conçu pour représenter l'activité de l'entreprise fictive DistriPlus Group, ainsi que la démarche suivie pour générer un jeu de données de test suffisamment volumineux pour rendre les mesures de performance significatives.

1.1 Contexte et choix technologiques

DistriPlus Group est une entreprise de distribution disposant de plusieurs magasins répartis sur le territoire, s'approvisionnant auprès de fournisseurs pour constituer un catalogue de produits, et vendant ces produits à une clientèle enregistrée. Ce contexte, volontairement simple, permet de couvrir les trois grandes familles d'opérations que l'on rencontre dans un système de gestion commerciale : l'enregistrement des ventes, la gestion du stock, et le suivi des approvisionnements.

Le projet a été réalisé sur Oracle Database 11g Express Edition, une édition gratuite d'Oracle limitée en ressources matérielles mais suffisante pour manipuler des volumes de plusieurs centaines de milliers de lignes. Le développement s'est fait principalement dans SQL Developer, l'environnement graphique officiel d'Oracle, complété ponctuellement par SQL*Plus pour certaines vérifications de session. Ce choix technologique correspond à celui imposé par le sujet du projet et présente l'intérêt d'être représentatif d'un environnement de développement Oracle standard, tout en restant accessible sans licence commerciale.

1.2 Modélisation de la base de données

Le schéma relationnel retenu comprend sept tables, chacune répondant à un besoin identifié dans les cas d'optimisation traités par la suite. Nous avons volontairement limité le schéma à ce nombre de tables plutôt que de chercher à représenter exhaustivement un système d'information réel, afin que chaque table ait un rôle clairement identifiable dans la suite du projet.

La table MAGASIN décrit les points de vente de l'entreprise. La table CLIENT et la table FOURNISSEUR décrivent respectivement la clientèle et les fournisseurs de produits. La table PRODUIT constitue le catalogue, chaque produit étant rattaché à un fournisseur et disposant d'un prix de référence. La table COMMANDE représente un acte de vente, rattaché à un magasin et à un client, et la table LIGNE_COMMANDE en détaille le contenu produit par

produit. Enfin, la table STOCK décrit la quantité disponible pour chaque produit dans chaque magasin.

MAGASIN	(<u>magasin_id</u> , nom, ville, region)
CLIENT	(<u>client_id</u> , nom, ville)
FOURNISSEUR	(<u>fournisseur_id</u> , nom, ville)
PRODUIT	(<u>produit_id</u> , #fournisseur_id, nom, categorie, prix_reference)
COMMANDE	(<u>commande_id</u> , #magasin_id, #client_id, date_vente, montant_total)
LIGNE_COMMANDE	(<u>ligne_id</u> , #commande_id, #produit_id, quantite, prix_unitaire)
STOCK	(<u>stock_id</u> , #magasin_id, #produit_id, quantite_stock, date_maj)

Figure 1 — Schéma relationnel simplifié des sept tables du projet

Deux attributs de ce schéma méritent d'être signalés dès ce chapitre, car ils prendront tout leur sens au chapitre 4. D'une part, la colonne `prix_unitaire` de la table `LIGNE_COMMANDE` duplique volontairement une information déjà présente dans `PRODUIT` : il s'agit d'une dénormalisation contrôlée, mise en place dès la conception, dont la justification et la mesure font l'objet du chapitre 4. D'autre part, la table `LIGNE_COMMANDE` ne dispose initialement d'aucun lien direct vers `MAGASIN`, ce qui oblige à passer par la table `COMMANDE` pour relier une ligne de commande à son magasin — situation qui sera également revisitée au chapitre 4.

Chaque table dispose d'une clé primaire numérique et les relations entre tables sont matérialisées par des contraintes de clé étrangère explicitement nommées, ce qui a permis, lors des manipulations nécessitant leur désactivation temporaire, de les identifier sans ambiguïté.

1.3 Génération du jeu de données de test

Un jeu de données trop restreint ne permet pas de mesurer d'écarts de performance significatifs, car l'ensemble des données tient alors en mémoire et les différences entre deux stratégies d'exécution deviennent négligeables. Nous avons donc porté une attention particulière au volume des données générées, avec pour objectif d'atteindre un ordre de grandeur représentatif d'une activité réelle sur plusieurs mois, voire plusieurs années.

Tableau 1 — Volumes de données générés pour chaque table du schéma

Table	Nombre de lignes	Remarque
MAGASIN	5	Un par région couverte
CLIENT	200	Clientèle simulée
FOURNISSEUR	10	Fournisseurs simulés

PRODUIT	30	Catalogue simulé
COMMANDE	500 000	Historique de ventes simulé sur deux ans
LIGNE_COMMANDE	1 000 000	Deux lignes en moyenne par commande
STOCK	600 000	Historique quotidien simulé du stock

Les données ont été générées par des blocs PL/SQL utilisant le paquet DBMS_RANDOM pour produire des valeurs aléatoires réalistes (villes, catégories de produits, quantités, dates), combiné à la clause CONNECT BY LEVEL pour générer un grand nombre de lignes sans recourir à une boucle procédurale ligne par ligne. Cette approche, dite ensembliste ou set-based, consiste à construire le jeu de résultat par une seule instruction SQL plutôt que par une répétition d'insertions unitaires ; elle s'est révélée nettement plus rapide que l'insertion via une boucle FOR, ce qui constitue d'ailleurs, par anticipation, une première illustration du principe qui sera développé au chapitre 3 à propos du traitement ensembliste en PL/SQL.

```

DECLARE
    v_batch_size NUMBER := 100000;
    v_nb_clients NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_nb_clients FROM client;
    FOR m IN 1..5 LOOP
        INSERT INTO commande (commande_id, magasin_id, client_id, date_vente,
montant_total)
        SELECT (m-1)*v_batch_size + ROWNUM, magasin_id, client_id, date_vente, NULL
        FROM (
            SELECT m AS magasin_id,
                   TRUNC(DBMS_RANDOM.VALUE(1, v_nb_clients + 1)) AS client_id,
                   DATE '2024-01-01' + TRUNC(DBMS_RANDOM.VALUE(0,730)) AS date_vente
            FROM dual
            CONNECT BY LEVEL <= v_batch_size
            ORDER BY date_vente
        );
        COMMIT;
    END LOOP;
END;
/

```

Figure 2 — Extrait du script de génération de la table COMMANDE (500 000 lignes)

Les commandes ont été générées magasin par magasin, chaque lot étant en outre trié par date de vente avant insertion. Ce choix, qui peut sembler anecdotique à ce stade, a une conséquence directe sur la disposition physique des données sur le disque, et donc sur l'efficacité d'un index qui sera créé au chapitre 3 sur les colonnes magasin_id et date_vente : en insérant les données déjà regroupées de cette manière, on obtient un bon facteur de regroupement, ou clustering factor, notion détaillée dans l'encart ci-dessous.

Tableau 2 — Le facteur de regroupement (*clustering factor*)

Notion	Explication
Clustering factor	Le facteur de clustering mesure si les lignes ayant la même valeur d'index sont stockées près les unes des autres sur le disque. Plus il est proche du nombre de blocs de la table, plus les données sont bien regroupées et l'index est efficace. En revanche, s'il est proche du nombre total de lignes, les données sont dispersées et l'utilisation de l'index est moins avantageuse pour l'optimiseur.

1.4 Vérification préalable de la base

Avant d'entreprendre toute optimisation, nous avons pris soin de vérifier la cohérence du schéma obtenu : nombre de lignes par table, présence des sept contraintes de clé étrangère attendues, et absence de tout index autre que ceux créés automatiquement par Oracle pour chaque clé primaire. Cette étape, bien que peu spectaculaire, correspond à une recommandation explicite du cadrage du projet : comprendre la logique métier et la structure de la base avant de chercher à l'optimiser. Elle nous a également permis de nous assurer que les mesures « avant optimisation » présentées dans les chapitres suivants partent toutes d'un état de référence identique et maîtrisé.

Chapitre 2 — Méthodologie de diagnostic des performances

Avant de présenter les optimisations proprement dites, il convient de détailler la méthode de diagnostic suivie tout au long du projet, qui repose sur trois éléments : la collecte préalable des statistiques Oracle, la lecture du plan d'exécution prévisionnel avec EXPLAIN PLAN, et la mesure réelle d'exécution avec AUTOTRACE, complétée si nécessaire par un chronométrage direct.

2.1 La collecte des statistiques

L'optimiseur Oracle, appelé Cost-Based Optimizer, choisit la stratégie d'exécution d'une requête en s'appuyant sur des statistiques décrivant chaque table et chaque index : nombre de lignes, distribution des valeurs, taille moyenne des lignes. Ces statistiques ne sont pas calculées automatiquement après une opération de chargement massif de données ; elles doivent être explicitement demandées par la commande DBMS_STATS.GATHER_SCHEMA_STATS, exécutée sur l'ensemble du schéma après chaque modification importante du volume de données.

```
BEGIN
  DBMS_STATS.GATHER_SCHEMA_STATS (
    ownname => 'DISTRIPUS',
    estimate_percent => DBMS_STATS.AUTO_SAMPLE_SIZE
  );
END;
/
```

Figure 3 — Collecte des statistiques du schéma DISTRIPUS

Sans cette étape, un plan d'exécution observé avec EXPLAIN PLAN n'aurait aucune valeur diagnostique, l'optimiseur travaillant alors sur des informations vides ou obsolètes. Cette collecte a donc été systématiquement effectuée avant chaque nouvelle série de mesures présentée dans ce mémoire.

2.2 Le plan d'exécution avec EXPLAIN PLAN

La commande EXPLAIN PLAN FOR, suivie de l'affichage du résultat par la fonction DBMS_XPLAN.DISPLAY, permet d'obtenir la stratégie que l'optimiseur prévoit d'employer pour exécuter une requête donnée, sans exécuter réellement cette requête. Le résultat se présente sous la forme d'un arbre d'opérations, chacune assortie d'un coût estimé et d'une cardinalité, c'est-à-dire du nombre de lignes que l'optimiseur pense devoir traiter à cette étape.

Trois types d'opérations reviennent régulièrement dans les plans observés au cours du projet. Le parcours complet d'une table, ou Full Table Scan, consiste à lire l'ensemble des blocs d'une table sans passer par un index ; il constitue généralement le signal d'une optimisation possible lorsqu'il porte sur une table volumineuse et qu'un filtre sélectif existe dans la requête. La jointure par hachage, ou Hash Join, construit une structure de hachage en mémoire à partir de l'une des deux tables jointes pour retrouver rapidement les lignes correspondantes de l'autre table ; elle est privilégiée par l'optimiseur lorsque les deux ensembles à joindre sont de taille importante. Le parcours d'index, ou Index Range Scan, permet au contraire de n'accéder qu'aux lignes réellement concernées par un filtre, à condition qu'un index existant corresponde à ce filtre.

Le coût affiché par EXPLAIN PLAN est une valeur abstraite de comparaison, propre à l'optimiseur, et ne doit pas être interprété comme un temps d'exécution en secondes. Il reste néanmoins l'indicateur le plus direct pour comparer deux versions d'une même requête, à condition que les statistiques utilisées soient à jour.

2.3 La mesure réelle avec AUTOTRACE

Contrairement à EXPLAIN PLAN, la fonctionnalité AUTOTRACE de SQL Developer exécute réellement l'instruction et fournit, en plus du plan d'exécution effectivement suivi, un ensemble de statistiques de session parmi lesquelles nous avons retenu deux indicateurs principaux. Le nombre de lectures logiques consistantes, ou consistent gets, mesure le nombre de blocs de données consultés en mémoire pour répondre à la requête ; c'est l'indicateur le plus fiable pour comparer deux versions d'une requête, car il ne dépend pas de la charge instantanée de la machine. Le nombre de lignes parcourues lors d'un parcours de table, table scan rows gotten, permet quant à lui d'objectiver très concrètement l'ampleur d'un parcours complet.

Ces deux outils ont été systématiquement utilisés ensemble dans ce projet : EXPLAIN PLAN pour identifier la stratégie choisie par l'optimiseur, AUTOTRACE pour vérifier que cette stratégie se traduit effectivement par un travail réduit lors de l'exécution.

2.4 Une méthode complémentaire : le chronométrage direct

Au cours du chapitre 4, nous avons rencontré une situation où le coût affiché par EXPLAIN PLAN et le nombre de lectures logiques mesuré par AUTOTRACE restaient quasiment identiques entre deux versions d'une même requête, alors qu'une différence de performance réelle existait bel et bien. Cette situation s'explique par le fait que ces deux indicateurs reflètent

avant tout un travail de lecture disque, alors que la différence observée provenait d'un travail de calcul processeur, invisible dans ces métriques. Nous avons alors complété la mesure par la commande SET TIMING, qui affiche le temps réel écoulé lors de l'exécution d'une instruction ou d'un bloc PL/SQL, ce qui a permis de révéler un écart que les deux outils précédents ne permettaient pas de mettre en évidence.

Cette méthode complémentaire a également été utilisée au chapitre 3 pour comparer deux versions d'une procédure PL/SQL, EXPLAIN PLAN ne s'appliquant pas directement à un bloc procédural complet mais seulement aux instructions SQL qu'il contient.

Chapitre 3 — Optimisation applicative

L'axe applicatif regroupe les optimisations qui améliorent la performance d'une requête ou d'un traitement sans modifier la structure des tables : réécriture d'une requête, création d'un index, ou réécriture d'un bloc PL/SQL. Deux cas ont été traités dans cet axe, retenus pour leur représentativité d'un système de gestion commerciale : une requête de restitution (le tableau de bord des ventes) et un traitement de calcul en masse (la valorisation du stock).

3.1 Cas 1 — La requête du tableau de bord des ventes

Le premier cas concerne une requête destinée à alimenter un tableau de bord de gestion, permettant à un responsable de magasin de consulter le chiffre d'affaires réalisé sur une période donnée.

3.1.1 État initial et diagnostic

Une première version de cette requête, agrégeant l'intégralité des ventes tous magasins et toutes dates confondues, a été analysée avec EXPLAIN PLAN. Le plan obtenu affichait un coût de 3401 et faisait apparaître un parcours complet des deux tables les plus volumineuses du schéma, COMMANDE et LIGNE_COMMANDE, suivi d'une jointure par hachage entre elles.

Id	Operation	Name	Rows	Cost (%CPU)
0	SELECT STATEMENT		2581	3401 (2)
1	SORT GROUP BY		2581	3401 (2)
* 2	HASH JOIN		2581	3400 (2)
3	TABLE ACCESS FULL	MAGASIN	5	2 (0)
6	HASH JOIN		987K	3372 (1)
7	TABLE ACCESS FULL	COMMANDE	500K	462 (1)
8	TABLE ACCESS FULL	LIGNE_COMMANDE	1000K	1010 (1)

Figure 4 — Plan d'exécution de la requête initiale (extrait)

Ce constat appelait une remarque méthodologique avant toute tentative d'optimisation : une requête qui agrège l'intégralité d'une table n'offre, par construction, aucune marge de progression par la voie d'un index, puisque la totalité des lignes doit de toute façon être lue. La véritable question n'était donc pas de savoir comment accélérer cette requête telle qu'elle était écrite, mais de vérifier si elle correspondait réellement au besoin métier. Un responsable de magasin consulte en pratique les ventes de son magasin sur une période donnée, non l'historique complet de l'ensemble du réseau ; nous avons donc reformulé la requête en conséquence.

3.1.2 Reformulation de la requête

```

SELECT m.nom AS magasin, c.date_vente, SUM(lc.quantite * lc.prix_unitaire) AS
total_vente
FROM commande c, ligne_commande lc, magasin m
WHERE c.commande_id = lc.commande_id
      AND c.magasin_id = m.magasin_id
      AND c.magasin_id = 2
      AND c.date_vente BETWEEN DATE '2024-06-01' AND DATE '2024-06-30'
GROUP BY m.nom, c.date_vente
ORDER BY c.date_vente;

```

Figure 5 — Requête reformulée avec filtre sur le magasin et la période

Cette seule reformulation a fait chuter le coût du plan à 1479, soit une réduction de 57 % par rapport à la version initiale, sans qu'aucune structure de la base n'ait été modifiée. Le plan obtenu montrait toutefois que la table COMMANDE, bien que filtrée à environ 4400 lignes sur les 500 000 qu'elle contient, était encore lue dans son intégralité : le filtre était appliqué après lecture complète, faute d'index correspondant.

3.1.3 Création d'un index composite

Ce constat justifiait la création d'un index composite portant sur les deux colonnes utilisées dans le filtre, magasin_id et date_vente. Un tel index permet à l'optimiseur d'accéder directement aux blocs concernés plutôt que de parcourir la table entière.

```

CREATE INDEX idx_commande_magasin_date
ON commande(magasin_id, date_vente);

```

Figure 6 — Création de l'index composite

Après recalcul des statistiques, le nouveau plan d'exécution faisait apparaître un parcours d'index (Index Range Scan) suivi d'un accès aux lignes correspondantes par identifiant physique (Table Access By Index Rowid), remplaçant le parcours complet observé précédemment. Le coût du plan est descendu à 1046, et le nombre de lectures logiques mesuré par AUTOTRACE est passé de 5398 à 3739.

3.1.4 Résultats et discussion

Tableau 3 — Évolution du coût et des lectures logiques pour le Cas 1

Étape	Cost	Consistent gets
Requête initiale, sans filtre	3401	5398
Requête filtrée, sans index composite	1479	-
Requête filtrée, avec index composite	1046	3739

Le gain global entre la version initiale et la version finale atteint 69 % sur le coût d'exécution et 31 % sur le nombre de lectures logiques. Ce résultat illustre un principe que nous retenons comme l'un des apports méthodologiques les plus importants du projet : la performance d'une requête ne se résume pas au choix d'un index, mais dépend d'abord de la pertinence de la requête elle-même au regard du besoin réel qu'elle sert. L'index, dans ce cas précis, n'a apporté un gain supplémentaire qu'une fois la requête correctement recentrée sur ce besoin.

Nous précisons enfin, pour la rigueur de l'analyse, que l'efficacité de cet index composite dépend directement du facteur de regroupement des données évoqué au chapitre 1 : les commandes ayant été générées groupées par magasin et triées par date, les lignes correspondant à un même magasin sont physiquement proches sur le disque, ce qui explique que l'optimiseur ait choisi d'utiliser l'index plutôt que de l'ignorer.

3.2 Cas 2 — La procédure de valorisation du stock

Le second cas de l'axe applicatif porte sur une procédure PL/SQL chargée de calculer la valeur totale du stock de l'entreprise, en multipliant pour chaque ligne de stock la quantité disponible par le prix de référence du produit concerné.

3.2.1 Version initiale

Une première version de cette procédure parcourt la table STOCK ligne par ligne à l'aide d'un curseur, et effectue pour chaque ligne une instruction SELECT INTO distincte afin de récupérer le prix de référence du produit correspondant dans la table PRODUIT.

```
CREATE OR REPLACE PROCEDURE valoriser_stock_naif AS
  v_prix NUMBER;
  v_valeur_totale NUMBER := 0;
BEGIN
  FOR ligne IN (SELECT produit_id, quantite_stock FROM stock) LOOP
    SELECT prix_reference INTO v_prix
    FROM produit
    WHERE produit_id = ligne.produit_id;

    v_valeur_totale := v_valeur_totale + (ligne.quantite_stock * v_prix);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Valeur totale du stock : ' || v_valeur_totale);
END;
/
```

Figure 7 — Procédure de valorisation du stock — version initiale

Sur la table STOCK, qui compte 600 000 lignes représentant un historique quotidien simulé, cette approche exécute 600 000 instructions SELECT distinctes, chacune constituant un aller-retour entre le moteur PL/SQL et le moteur SQL sous-jacent. Chronométrée avec SET TIMING, cette procédure s'exécute en 8,82 secondes.

3.2.2 Version optimisée par traitement ensembliste

La version optimisée remplace entièrement cette boucle par une unique requête de jointure entre STOCK et PRODUIT, dont le résultat est récupéré en une seule fois en mémoire grâce à l'instruction BULK COLLECT.

```
CREATE OR REPLACE PROCEDURE valoriser_stock_optimise AS
  TYPE t_valeur IS TABLE OF NUMBER;
  v_valeurs t_valeur;
  v_valeur_totale NUMBER := 0;
BEGIN
  SELECT s.quantite_stock * p.prix_reference
  BULK COLLECT INTO v_valeurs
  FROM stock s, produit p
  WHERE s.produit_id = p.produit_id;

  FOR i IN 1..v_valeurs.COUNT LOOP
    v_valeur_totale := v_valeur_totale + v_valeurs(i);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('Valeur totale du stock (optimisée) : ' ||
v_valeur_totale);
END;
/
```

Figure 8 — Procédure de valorisation du stock — version optimisée

BULK COLLECT permet de récupérer en une seule opération l'ensemble des lignes résultant d'une requête, au lieu de les parcourir une à une par un curseur implicite. Combinée à la jointure directe entre STOCK et PRODUIT, cette approche ramène le calcul du prix à une seule instruction SQL, quel que soit le nombre de lignes de STOCK. La boucle qui subsiste dans le code ne fait plus qu'additionner des valeurs déjà présentes en mémoire, ce qui ne représente plus qu'un coût de calcul négligeable.

3.2.3 Résultats et discussion

Tableau 4 — Comparaison des temps d'exécution pour le Cas 2

Version	Temps d'exécution	Gain
Naïve (curseur + SELECT répété, 600 000 fois)	8,82 s	référence
Optimisée (BULK COLLECT + jointure directe)	0,40 s	≈ 22 fois plus rapide

Le gain mesuré, de l'ordre de 95 %, s'explique entièrement par la suppression des allers-retours répétés entre le moteur procédural et le moteur SQL. Chaque instruction SELECT, même très rapide isolément, comporte un coût fixe qui devient significatif lorsqu'il est répété plusieurs centaines de milliers de fois ; en remplaçant cette répétition par une opération unique traitant l'ensemble des données, le traitement ensembliste supprime ce coût structurel plutôt que de simplement l'accélérer marginalement.

Ce cas illustre un principe général de la programmation PL/SQL sur Oracle, souvent résumé par l'expression « privilégier le traitement par ensemble au traitement ligne par ligne », que nous avons d'ailleurs déjà rencontré, sans le nommer explicitement, lors de la génération du jeu de données présentée au chapitre 1.

Chapitre 4 — Optimisation de la modélisation

Contrairement à l'axe applicatif présenté au chapitre précédent, l'axe modélisation agit sur la structure même des tables : il s'agit ici de dupliquer volontairement une information dans le schéma, choix communément appelé dénormalisation contrôlée, afin d'éviter un traitement coûteux lors de la lecture. Deux cas ont été retenus, se distinguant par la nature de l'information dupliquée : un attribut métier dans le premier cas, une clé étrangère dans le second.

4.1 Cas 1 — Dénormalisation contrôlée du prix produit

Le schéma présenté au chapitre 1 comporte, dans la table `LIGNE_COMMANDE`, une colonne `prix_unitaire` qui duplique une information a priori déjà disponible dans la table `PRODUIT` par l'intermédiaire de la colonne `prix_reference`. Ce choix, mis en place dès la conception du schéma, répond à une exigence métier précise : le prix facturé au client au moment d'une vente doit rester figé dans le temps, même si le prix catalogue du produit évolue ultérieurement. Sans cette duplication, toute modification du prix de référence d'un produit modifierait rétroactivement le montant de ventes déjà enregistrées, ce qui serait incorrect du point de vue comptable.

4.1.1 Comparaison avec et sans dénormalisation

Pour mesurer l'intérêt de ce choix, nous avons comparé deux requêtes calculant le chiffre d'affaires total sur l'ensemble de la table `LIGNE_COMMANDE`, soit à partir du prix dénormalisé, soit en le recalculant par une jointure avec la table `PRODUIT`.

```
SELECT SUM(quantite * prix_unitaire) AS chiffre_affaires
FROM ligne_commande;
```

Figure 9 — Requête avec dénormalisation (prix déjà présent dans `LIGNE_COMMANDE`)

```
SELECT SUM(lc.quantite * p.prix_reference) AS chiffre_affaires
FROM ligne_commande lc, produit p
WHERE lc.produit_id = p.produit_id;
```

Figure 10 — Requête équivalente sans dénormalisation (jointure avec `PRODUIT`)

L'examen du plan d'exécution de ces deux requêtes fait apparaître une différence structurelle nette : la première ne lit que la table `LIGNE_COMMANDE`, tandis que la seconde nécessite une jointure supplémentaire avec la table `PRODUIT`. Le coût affiché par `EXPLAIN PLAN` reste toutefois quasiment identique entre les deux versions, 1012 contre

1015, ce qui s'explique par la taille très réduite de la table PRODUIT, trente lignes seulement dont la lecture complète ne représente qu'une charge négligeable en termes de blocs disque.

Cette observation nous a conduits, conformément à la méthode présentée au chapitre 2, à mesurer également le temps réel d'exécution des deux requêtes sur l'intégralité du million de lignes de LIGNE_COMMANDE.

Tableau 5 — Comparaison des temps d'exécution pour le Cas 1 de modélisation

Version	Temps d'exécution	Gain
Avec dénormalisation (prix déjà dans LIGNE_COMMANDE)	0,13 s	référence
Sans dénormalisation (jointure avec PRODUIT)	0,31 s	2,4 fois plus lent

4.1.2 Discussion

Cet écart, bien que modeste dans l'absolu, s'explique par un mécanisme précis : la version sans dénormalisation doit construire et interroger une structure de hachage pour chacune des lignes de LIGNE_COMMANDE, opération purement processeur qui n'apparaît ni dans le coût d'EXPLAIN PLAN ni dans le nombre de lectures logiques d'AUTOTRACE, ces deux indicateurs mesurant essentiellement un travail d'entrées-sorties disque. Ce cas illustre ainsi une limite importante des outils de diagnostic présentés au chapitre 2 : un gain de performance réel peut exister sans se traduire par une différence visible sur les métriques habituelles, lorsque ce gain relève d'un travail de calcul plutôt que de lecture.

La dénormalisation du prix présente en contrepartie un inconvénient qu'il convient de mentionner : la valeur dupliquée doit être maintenue cohérente au moment de l'insertion d'une ligne de commande, ce qui déplace la responsabilité de cette cohérence vers le code applicatif. Ce compromis entre performance de lecture et complexité de maintenance en écriture est caractéristique de toute décision de dénormalisation.

4.2 Cas 2 — Redondance contrôlée d'une clé étrangère

Le second cas de modélisation porte non plus sur un attribut métier, mais sur une relation entre tables. Dans le schéma initial, relier une ligne de commande à son magasin nécessite de passer par la table COMMANDE, LIGNE_COMMANDE ne disposant d'aucun lien direct vers MAGASIN. Cette situation impose une jointure transitive systématique pour toute requête souhaitant analyser les lignes de commande par magasin.

4.2.1 Requête avant modification

```
SELECT c.magasin_id, SUM(lc.quantite) AS quantite_totale
FROM ligne_commande lc, commande c
WHERE lc.commande_id = c.commande_id
      AND lc.produit_id = 5
GROUP BY c.magasin_id;
```

Figure 11 — Requête nécessitant une jointure transitive via *COMMANDE*

Le plan d'exécution de cette requête affiche un coût de 1478 et fait apparaître une jointure par hachage entre *LIGNE_COMMANDE* et *COMMANDE*, cette dernière étant lue intégralement.

4.2.2 Ajout de la clé étrangère redondante

Nous avons ajouté à la table *LIGNE_COMMANDE* une colonne *magasin_id*, contrainte par une clé étrangère vers *MAGASIN*, et alimentée par report de la valeur déjà présente dans la commande correspondante.

```
ALTER TABLE ligne_commande ADD magasin_id NUMBER;

UPDATE ligne_commande lc
SET magasin_id = (
  SELECT c.magasin_id FROM commande c WHERE c.commande_id = lc.commande_id
);

ALTER TABLE ligne_commande ADD CONSTRAINT fk_ligne_magasin
FOREIGN KEY (magasin_id) REFERENCES magasin(magasin_id);
```

Figure 12 — Ajout de la colonne redondante *magasin_id*

La requête équivalente peut désormais s'écrire sans aucune jointure, *magasin_id* étant directement disponible sur *LIGNE_COMMANDE*.

```
SELECT magasin_id, SUM(quantite) AS quantite_totale
FROM ligne_commande
WHERE produit_id = 5
GROUP BY magasin_id;
```

Figure 13 — Requête après ajout de la clé étrangère redondante

4.2.3 Résultats et discussion

Tableau 6 — Comparaison avant/après pour le Cas 2 de modélisation

Indicateur	Avant (jointure via <i>COMMANDE</i>)	Après (<i>magasin_id</i> direct)
Cost (EXPLAIN PLAN)	1478	1014

Consistent gets (AUTOTRACE)	5401	3709
Table scan rows gotten	1 500 426	1 000 426

Le gain atteint 31 % à la fois sur le coût d'exécution et sur le nombre de lectures logiques. La lecture du nombre de lignes parcourues est particulièrement éclairante : la différence entre les deux mesures, 500 000 lignes, correspond exactement au nombre de lignes de la table COMMANDE, confirmant sans ambiguïté que le gain provient de la suppression complète de cette table du plan d'exécution.

Ce cas se distingue du précédent par la nature de ce qui est dénormalisé : il ne s'agit plus de dupliquer une valeur métier soumise à une exigence d'historique, mais de dupliquer une relation entre deux tables, afin de raccourcir un chemin de navigation. L'inconvénient associé diffère également : si une commande venait à changer de magasin après sa création, il faudrait répercuter ce changement sur toutes les lignes de commande associées pour conserver la cohérence de la duplication.

Chapitre 5 — Synthèse comparative des optimisations

Ce chapitre rassemble les résultats obtenus pour les quatre cas traités dans ce mémoire et propose une lecture transversale des enseignements méthodologiques qui en découlent.

5.1 Tableau de synthèse

Tableau 7 — Synthèse des quatre cas d'optimisation traités

Axe	Cas	Méthode de mesure	Gain obtenu
Applicatif	Requête du tableau de bord des ventes	EXPLAIN PLAN / AUTOTRACE	Coût -69 %, lectures logiques -31 %
Applicatif	Procédure de valorisation du stock	SET TIMING	Temps d'exécution , - 95 %
Modélisation	Dénormalisation du prix produit	SET TIMING	Temps d'exécution , - 58 %
Modélisation	Redondance d'une clé étrangère	EXPLAIN PLAN / AUTOTRACE	Coût -31 %, lectures logiques -31 %

5.2 Enseignements transversaux

Le premier enseignement de ce projet concerne le choix de l'outil de mesure. Deux des quatre cas ont pu être évalués directement par le coût d'exécution et le nombre de lectures logiques, ces indicateurs reflétant fidèlement une réduction du travail de lecture disque. Les deux autres cas, en revanche, impliquaient une table jointe de très petite taille, pour laquelle ces mêmes indicateurs restaient quasiment inchangés malgré un gain réel : il a fallu, dans ces deux situations, recourir à un chronométrage direct pour objectiver la différence. Cette expérience nous a appris qu'un outil de diagnostic, quelle que soit sa pertinence générale, doit être choisi en fonction de la nature du travail que l'on cherche à mesurer, travail d'entrées-sorties dans un cas, travail de calcul dans l'autre.

Le second enseignement porte sur la distinction entre les deux axes d'optimisation. Les deux cas applicatifs ont montré qu'une amélioration de performance peut être obtenue sans toucher à la structure des tables, par une simple reformulation de requête ou de procédure, éventuellement complétée par un index. Les deux cas de modélisation ont montré à l'inverse qu'une duplication contrôlée d'information, décidée dès la conception du schéma ou introduite ultérieurement, peut supprimer entièrement le besoin d'une jointure, au prix d'une charge de maintenance supplémentaire en écriture. Ces deux approches ne sont pas concurrentes mais

complémentaires, comme l'illustre le premier cas applicatif, où le gain obtenu par la reformulation de la requête et celui obtenu par la création de l'index composite se sont cumulés.

Enfin, le cas de l'index composite a mis en évidence l'importance du facteur de regroupement physique des données, notion rarement abordée dans un cours d'introduction aux bases de données mais déterminante en pratique : un index parfaitement pertinent sur le plan logique peut être ignoré par l'optimiseur si les données qu'il indexe sont physiquement dispersées sur le disque.

Conclusion et perspectives

Bilan de la problématique

Ce projet nous a permis de traiter, sur un cas concret et à volume représentatif, la question posée en introduction : dans quelle mesure une modification ciblée du code applicatif ou du modèle de données permet-elle d'améliorer mesurablement les performances d'une base Oracle. Les quatre cas présentés dans ce mémoire y répondent chacun à leur manière, avec des gains allant de 31 % à 95 % selon la nature de l'optimisation, et surtout avec, pour chacun, une explication du mécanisme sous-jacent qui va au-delà du simple constat d'un chiffre amélioré.

Ce qu'il reste à approfondir

Le périmètre de ce projet a volontairement exclu l'étude de l'architecture physique de la base, répartition des tablespaces, séparation des fichiers de journalisation, cet axe ayant été retiré du sujet en cours de projet en accord avec notre encadrant, la configuration matérielle disponible ne permettant pas d'en tirer des mesures probantes. Un prolongement naturel de ce travail consisterait à reprendre cette question sur une infrastructure disposant de plusieurs disques physiques distincts.

Compétences acquises

Sur le plan technique, ce projet nous a permis de manipuler concrètement les outils de diagnostic Oracle EXPLAIN PLAN et AUTOTRACE, de mettre en œuvre des techniques d'optimisation PL/SQL telles que BULK COLLECT, et de comprendre par la pratique des notions généralement présentées de façon abstraite en cours, comme le facteur de regroupement d'un index ou la distinction entre coût logique et temps réel d'exécution.

Sur le plan méthodologique, nous retenons surtout l'importance de mesurer avant d'agir, puis de mesurer à nouveau après toute modification, plutôt que de se fier à une intuition sur ce qui devrait fonctionner. C'est cette discipline de mesure systématique, plus que la connaissance d'une technique d'optimisation en particulier, qui constitue selon nous l'apport principal de ce projet pour notre pratique future.

Perspectives

Au-delà du cadre de ce projet académique, la démarche suivie ici : diagnostiquer avant d'optimiser, choisir l'outil de mesure adapté à la nature du gain recherché, documenter le compromis associé à chaque décision de modélisation nous semble directement transposable à un contexte professionnel, où les volumes de données et les enjeux de performance sont généralement plus importants encore que ceux simulés dans ce mémoire.

Annexe A — Script complet de création du schéma

Cette annexe reproduit l'intégralité des instructions de création des sept tables du schéma DISTRIPPLUS, dans l'ordre imposé par les dépendances de clés étrangères.

```
CREATE TABLE magasin (  
    magasin_id    NUMBER PRIMARY KEY,  
    nom           VARCHAR2(100) NOT NULL,  
    ville         VARCHAR2(100),  
    region        VARCHAR2(50)  
);  
  
CREATE TABLE client (  
    client_id     NUMBER PRIMARY KEY,  
    nom           VARCHAR2(100) NOT NULL,  
    ville         VARCHAR2(100)  
);  
  
CREATE TABLE fournisseur (  
    fournisseur_id NUMBER PRIMARY KEY,  
    nom           VARCHAR2(100) NOT NULL,  
    ville         VARCHAR2(100)
```

```

);

CREATE TABLE produit (
    produit_id      NUMBER PRIMARY KEY,
    nom             VARCHAR2(150) NOT NULL,
    categorie       VARCHAR2(50),
    prix_reference  NUMBER(10,2),
    fournisseur_id  NUMBER,
    CONSTRAINT fk_produit_fournisseur FOREIGN KEY (fournisseur_id)
        REFERENCES fournisseur(fournisseur_id)
);

CREATE TABLE commande (
    commande_id     NUMBER PRIMARY KEY,
    magasin_id      NUMBER NOT NULL,
    client_id       NUMBER,
    date_vente      DATE NOT NULL,
    montant_total   NUMBER(10,2),
    CONSTRAINT fk_commande_magasin FOREIGN KEY (magasin_id)
        REFERENCES magasin(magasin_id),
    CONSTRAINT fk_commande_client FOREIGN KEY (client_id)
        REFERENCES client(client_id)
);

CREATE TABLE ligne_commande (
    ligne_id        NUMBER PRIMARY KEY,
    commande_id     NUMBER NOT NULL,
    produit_id      NUMBER NOT NULL,
    quantite        NUMBER NOT NULL,
    prix_unitaire   NUMBER(10,2) NOT NULL,
    CONSTRAINT fk_ligne_commande FOREIGN KEY (commande_id)
        REFERENCES commande(commande_id),
    CONSTRAINT fk_ligne_produit FOREIGN KEY (produit_id)
        REFERENCES produit(produit_id)
);

CREATE TABLE stock (
    stock_id        NUMBER PRIMARY KEY,
    magasin_id      NUMBER NOT NULL,
    produit_id      NUMBER NOT NULL,
    quantite_stock  NUMBER DEFAULT 0,
    date_maj        DATE,
    CONSTRAINT fk_stock_magasin FOREIGN KEY (magasin_id)
        REFERENCES magasin(magasin_id),
    CONSTRAINT fk_stock_produit FOREIGN KEY (produit_id)
        REFERENCES produit(produit_id)
);

```

Figure 14 — Création des sept tables du schéma

Annexe B — Script de génération du jeu de données

Cette annexe reproduit le script de génération de la table STOCK, illustrant la technique de génération ensembliste par CONNECT BY LEVEL décrite au chapitre 1.

```

INSERT INTO stock (stock_id, magasin_id, produit_id, quantite_stock, date_maj)
SELECT ROWNUM, magasin_id, produit_id,
       TRUNC(DBMS_RANDOM.VALUE(0,500)),
       DATE '2014-01-01' + jour - 1
FROM (
  SELECT m.magasin_id, p.produit_id, j.jour
  FROM (SELECT LEVEL AS magasin_id FROM dual CONNECT BY LEVEL <= 5) m,
       (SELECT LEVEL AS produit_id FROM dual CONNECT BY LEVEL <= 30) p,
       (SELECT LEVEL AS jour FROM dual CONNECT BY LEVEL <= 4000) j
);
COMMIT;

```

Figure 15 — Génération de la table STOCK (600 000 lignes)

Lexique

Ce lexique regroupe, par ordre alphabétique, les principaux termes techniques employés dans ce rapport, définis de façon simple pour un lecteur non spécialiste.

AUTOTRACE : Fonctionnalité de SQL Developer et SQL*Plus qui exécute réellement une instruction SQL et affiche, en complément de son plan d'exécution, des statistiques de session telles que le nombre de lectures logiques.

Bulk Collect : Instruction PL/SQL qui récupère en une seule opération l'ensemble des lignes résultant d'une requête, dans une structure en mémoire, plutôt que de les parcourir une à une par un curseur.

Clustering factor (facteur de regroupement) : Indicateur mesurant la proximité physique, sur le disque, des lignes partageant une même valeur indexée. Un facteur élevé signale des données dispersées, rendant un index moins avantageux pour l'optimiseur.

Cost (coût) : Valeur numérique abstraite calculée par l'optimiseur Oracle pour comparer différentes stratégies d'exécution possibles d'une même requête. Ce n'est pas une mesure de temps.

Dénormalisation : Choix de conception consistant à dupliquer volontairement une information dans plusieurs tables, afin d'éviter une jointure coûteuse, au prix d'un risque d'incohérence à gérer.

EXPLAIN PLAN : Commande Oracle qui affiche la stratégie d'exécution prévue par l'optimiseur pour une requête donnée, sans exécuter réellement cette requête.

Full Table Scan : Opération consistant à lire l'intégralité des blocs d'une table, sans passer par un index.

Hash Join (jointure par hachage) : Méthode de jointure dans laquelle Oracle construit une structure de hachage à partir d'une des tables pour retrouver rapidement les lignes correspondantes de l'autre table.

Index : Structure de données auxiliaire permettant à Oracle d'accéder rapidement aux lignes d'une table satisfaisant une condition, sans en parcourir la totalité.

Index Range Scan : Parcours d'un index limité à une plage de valeurs correspondant à une condition de filtre.

Nested Loop (boucle imbriquée) : Méthode de jointure consistant à parcourir une première table puis, pour chaque ligne obtenue, rechercher les lignes correspondantes dans la seconde table.

Oracle Database : SGBD relationnelle développé par Oracle.

PL/SQL : Langage procédural propriétaire d'Oracle, qui étend le langage SQL par des structures de contrôle telles que les boucles et les conditions.

SET TIMING : Commande de SQL*Plus et SQL Developer qui affiche le temps réel écoulé lors de l'exécution d'une instruction ou d'un bloc PL/SQL.

SQL Developer : Environnement de développement graphique gratuit fourni par Oracle pour interagir avec une base de données Oracle.

Statistiques (DBMS_STATS) : Informations décrivant le volume et la distribution des données d'une table ou d'un index, utilisées par l'optimiseur pour choisir une stratégie d'exécution.

Bibliographie — Webographie

[1] Oracle Corporation, Oracle Database Concepts, 11g Release 2, Oracle Documentation, consulté en 2026.

[2] Oracle Corporation, Oracle Database SQL Tuning Guide, 11g Release 2, Oracle Documentation, consulté en 2026.

[3] Oracle Corporation, Oracle Database PL/SQL Language Reference, 11g Release 2, Oracle Documentation, consulté en 2026.

[4] Oracle Corporation, Oracle Database XE Documentation, <https://docs.oracle.com/database/xe/> consulté en 2026.